```
{
    "id": 2,
    "full_name": "Phoebe Jacobs",
    "birth": "1919-06-21",
    "death": "2012-04-09"
},
{
    "id": 3,
    "full_name": "Harvey Goldsmith",
    "birth": "1946-03-04",
    "death": null
},
{
    "id": 4,
    "full_name": "Allan Williams",
    "birth": "1930-02-21",
    "death": "2106-12-30"
}]
```

As I write this sentence, Harvey Goldsmith is still alive and, therefore, doesn't have a death date (date of passing? expiration date? period-of-pushing-up-petunias?). Where Django uses Python's `None` value, JavaScript, and therefore JSON, uses `null`.

> **A shortcut you shouldn't use**
>
> Similar to the `ModelForm` class, `ModelSchema` supports two values that can save you typing in a long list of fields. You can set the `fields` property to `"__all__"` to include all the fields from a Model, or instead of using the `fields` property, you can use an `exclude` property.
>
> Like with `ModelForm`, this isn't recommended. Explicitly listing your fields is safer. If you add a new field to your Model that shouldn't be serialized, it is better for the default to be exclusion. Inclusion, by default, means having to remember to edit the `ModelSchema` subclass to explicitly remove the new field.
>
> Not all fields get used in all situations. For example, if you have a creation timestamp on your object that automatically gets set by Django, you might want it serialized when reading an object but not when updating it. You shouldn't be able to change the creation timestamp through the API. This can mean needing different serializers for reading and updating—more on this later.

### 11.2.2 Handling arguments

Ninja supports both URL query parameters and URL-based arguments. Typically, query parameters get used to modify a result, while URL-based arguments get used to identify a target object. It is quite common for an API to provide two GET methods for each target object, one that lists objects and the other to fetch a specific object. This is akin to the `musician()` and `musicians()` views you wrote in chapter 4.

You've already seen the `promoters()` endpoint, so let's add a new one to fetch a single `Promoter` object. The difference, in this case, is the URL must support an